

Лабораторная работа №1

Разработка структуры проекта

Дисциплина: МДК.02.02 Инструментальные средства разработки ПО
Специальность: 09.02.07 Информационные системы и программирование
Студент: Радин Е. Д. (группа ИСПП-31)
Стек: Python 3.12 (FastAPI, SQLite, Pytest), HTML5, CSS3, Vanilla JS, Node.js
Репозиторий проекта: <https://github.com/RuttiTuttii/ISPRO-LabWork1>

1. Цель работы

- 1.1 Изучить процесс создания структуры проекта.
- 1.2 Научиться применять специализированное ПО для проектирования БД и интерфейса пользователя.

2. Проектирование структуры и базы данных (п. 5.1)

В рамках работы спроектирована реляционная база данных для образовательной платформы. Схема подготовлена в строгой черно-белой нотации ГОСТ / Crow's Foot шрифтом Times New Roman. Схема включает 12 нормализованных таблиц:

Таблица	Первичный / Внешний ключ	Назначение и состав полей
roles	PK id	Роли пользователей в системе (student, teacher)
users	PK id, FK role_id	Учетные записи пользователей, хэши паролей, ФИО
topics	PK id	Разделы учебного курса по архитектуре ПО
lectures	PK id, FK topic_id	Материалы лекций (HTML, видео, Markdown, файлы)
lecture_progress	PK id, FK user_id, FK lecture_id	Учет факта посещения и реального времени на странице
test_pools	PK id, FK topic_id	Контрольные тесты (банк 20 вопросов, выборка 10, лимит 600 с)
test_questions	PK id, FK pool_id	Тексты вопросов с подробными объяснениями ответов

test_options	PK id, FK question_id	По 4 варианта ответа на вопрос с указанием верного
test_attempts	PK id, FK user_id, FK pool_id	Итоговые баллы, проценты и академическая оценка
test_attempt_answers	PK id, FK attempt_id, FK question_id	Фиксация каждого выбранного студентом варианта
interactive_exercises	PK id, FK topic_id	Задания: ordering (DnD), grouping (DnD), regex
exercise_attempts	PK id, FK user_id, FK exercise_id	История выполнения интерактива и журнал ошибок

3. Реализация серверной и клиентской частей (п. 5.2 – 5.5)

1. Серверный модуль (Python / FastAPI):

- Полный CRUD теоретических материалов с поддержкой rich HTML, встроенного видео и прикрепленных файлов.
- Heartbeat-эндпоинт /api/lectures/{id}/progress для точного посекундного учета времени чтения.
- Генератор контрольного тестирования: случайная выборка ровно 10 вопросов из пула 20 без повторов, перемешивание порядка 4 вариантов ответов.
- Модуль оценивания тестирования по строгой регламентной шкале:
 - 90% – 100%: «отлично»;
 - 75% – 89.99%: «хорошо»;
 - 60% – 74.99%: «удовлетворительно»;
 - менее 60%: «неудовлетворительно».
- Модуль валидации интерактивных заданий (DnD сортировка этапов CI/CD, группировка паттернов GoF, проверка регулярных выражений SemVer) с системой подсказок и разбором ошибок.

2. Клиентский модуль (Frontend SPA):

- Выдержан в минималистичном бежево-черно-белом стиле (#F7F4EE, #FFFFFF, #1E1B18).
- Скругленные кнопки, полностью переназначенные кастомные стили выпадающих списков (select), полей ввода и радиокнопок.
- Глобальный скроллбар (:: -webkit-scrollbar) переназначен под цвет темного акцента.
- Интерактивный таймер чтения и таймер обратного отсчета для тестирования.
- Автоматизация запуска через корневой npm run dev / run.js без сторонних утилит.

4. Ответы на контрольные вопросы

8.1 Что такое «структура проекта»?

Структура проекта — это согласованная логическая и физическая организация файлов, каталогов, модулей, пакетов и пространств имен в кодовой базе, определяющая границы подсистем, правила взаимодействия между ними и направление потоков зависимостей. Она обеспечивает разделение ответственности (Separation of Concerns), контролирует связность компонентов (Cohesion и Coupling) и задает стандарты сборки, тестирования и поставки программного обеспечения.

8.2 Какие варианты структуры проекта используются при разработке современного ПО?

- **Монолитная слоистая (Layered / N-Tier):** разделение на Presentation, Business Logic и Data Access слои.
- **Чистая архитектура (Clean / Hexagonal):** изоляция предметной области от фреймворков и БД через инверсию зависимостей.
- **Модульный монолит (Modular Monolith):** единое приложение, разбитое на изолированные бизнес-модули.
- **Микросервисная архитектура (Microservices):** автономные сервисы со своими БД и независимым развертыванием.
- **Feature-based:** организация каталогов по бизнес-фичам вместо технических слоев.

8.3 Какие принципы используются при проектировании структуры проекта?

- **Single Responsibility Principle (SRP):** один класс / модуль отвечает за одну задачу.
- **Dependency Inversion Principle (DIP):** зависимости направлены к абстракциям, а не к деталям.
- **High Cohesion / Low Coupling:** высокая внутренняя связность и слабая внешняя зависимость.
- **Don't Repeat Yourself (DRY):** вынесение общих функций в модуль core.
- **Separation of Concerns (SoC):** отделение UI от хранения данных и бизнес-правил.

8.4 Какие риски возникают при непродуманной структуре проекта и как их избежать?

- Риск 1: превращение в «Big Ball of Mud» с хаотичными циклическими зависимостями.
- Риск 2: экспоненциальный рост трудоемкости доработок и регрессионные ошибки.
- Риск 3: невозможность изолированного тестирования.

Способы решения: проектирование архитектуры до кодинга, использование интерфейсов, статический анализ зависимостей в CI/CD и регулярный рефакторинг.

5. Вывод

В ходе работы была успешно спроектирована и реализована многоуровневая структура проекта образовательной веб-платформы. Освоены практические навыки построения ERD-схем и wireframe-макетов в draw.io по ГОСТ, реализован сервер на FastAPI и клиент на нативных веб-технологиях с бежево-черно-белой минималистичной эстетикой и скриптом `npm run dev`.